CodeIguanas

Cahier des charges

Application HomeSkolar

Spécifications fonctionnelles

1. **Gestion des utilisateurs (élève / tuteur / admin) :**

- ✓ Création de compte
- ✓ Connexion / déconnexion
- ✓ Gestion du mot de passe (modification, récupération)
- ✓ Modification des informations personnelles
- ✓ Gestion des rôles (définir qui est élève, tuteur ou admin)
- ✓ Assignation automatique des élève à un tuteur par l'admin
- ✓ Suivi des utilisateurs (l'admin gère le site, les comptes et garantir que chaque élève est suivi par un tuteur)

2. **Communication :**

- ✓ Envoyer et recevoir des messages
- ✓ Voir les messages non lus (notifications)
- ✓ Épingler des messages importants
- ✓ Consulter l'historique des conversations

3. **Gestion des rencontres :**

- ✓ Visualiser un calendrier (évènements et rendez-vous)
- ✓ Planifier un évènement ou un rendez-vous
- ✓ Modifier ou supprimer un évènement ou un rendez-vous
- ✓ Voir les disponibilités

4. **Gestion des tâches :**

- ✓ Créer une tâche après un rendez-vous
- ✓ Assigner une tâche à un élève
- ✓ Créer une tâche personnelle (mémo)
- ✓ Voir la liste des tâches
- ✓ Marquer une tâche comme terminée
- ✓ Recevoir des notifications de rappel

Veille technologique

Pour choisir les technologies du projet et assurer leur maintien dans le temps, nous avons réalisé une veille technologique en suivant le principe **KISS** et en favorisant des solutions simples, robustes et documentées avec une communauté très active.

1. **Outil :** Feedly

   - ✓ **Usage :** centraliser les flux RSS des technologies utilisées (React, Java, Spring Boot, MongoDB) pour rester informé en continu.
   - ✓ **Intérêt :** permet de repérer rapidement les nouveautés, changements ou problèmes potentiels et d'adapter la solution avant que cela n'impacte le projet.

2. **React (Front-end)**

   - ✓ **Source :** Site officiel React
   - ✓ **Pourquoi cette source :** La documentation officielle fournit toutes les bonnes pratiques, les mises à jour et des tutoriels fiables.
   - ✓ **Intérêt :** Suivre React via Feedly permet de rester informé des nouvelles fonctionnalités, des mises à jour et des retours de la communauté.
   - ✓ **Justification :** React assure un front-end dynamique et réactif, essentiel pour la messagerie, le calendrier et le suivi des tâches.

3. **Java + Spring Boot (Back-end)**

   - ✓ **Source :** Documentation officielle Spring Boot (spring.io) et OpenJDK
   - ✓ **Pourquoi cette source :** La documentation officielle Spring Boot fournit les guides techniques, les exemples de code et les meilleures pratiques pour créer des API REST robustes en Java.
   - ✓ **Intérêt :** Surveiller Spring Boot via Feedly permet de suivre les évolutions de sécurité, les nouvelles versions et les bonnes pratiques de la communauté Java.
   - ✓ **Justification :** Java + Spring Boot offre une architecture orientée objet robuste, alignée avec le diagramme de classes UML du projet. Spring Security permet la gestion des rôles et de l'authentification, et peut être configuré pour utiliser JWT via une implémentation personnalisée.

4. **MongoDB (Base de données NoSQL)**

- ✓ **Source :** Documentation officielle MongoDB
- ✓ **Pourquoi cette source :** Documentation complète et officielle pour la configuration, les bonnes pratiques et les nouveaux outils.
- ✓ **Intérêt :** Feedly permet de recevoir les actualités sur MongoDB, les mises à jour de version et les cas d'usage pertinents.
- ✓ **Justification :** MongoDB offre flexibilité et scalabilité pour stocker les utilisateurs, messages, rendez-vous et tâches. Adaptée aux évolutions futures de HomeSkolar.

Spécifications techniques

Cette section décrit les choix techniques retenus pour répondre aux besoins du projet HomeSkolar, ainsi que leur rôle dans l'application. Les technologies ont été sélectionnées selon leur simplicité d'utilisation, leur maintenabilité et leur adéquation avec les fonctionnalités attendues.

L'architecture globale repose sur une séparation claire entre le front-end (interface utilisateur) et le back-end (logique métier et gestion des données), communiquant via une API REST.

1. Front-end – React

Rôle : Le front-end est responsable de l'affichage des pages et de l'interaction avec l'utilisateur (élève, tuteur ou admin).

Technologie choisie : React

Pourquoi ce choix : React a été choisi pour sa capacité à créer des interfaces dynamiques et réactives grâce à une architecture basée sur des composants réutilisables.

Réponse aux besoins :

- ✓ Affichage en temps réel des messages
- ✓ Mise à jour dynamique du calendrier et des tâches
- ✓ Interface fluide pour améliorer l'expérience utilisateur

Avantages :

- ✓ Réutilisation des composants (maintenance facilitée)
- ✓ Grande communauté et documentation riche
- ✓ Performances optimisées grâce au Virtual DOM

Pourquoi pas une autre solution : Comparé à des solutions plus lourdes ou moins populaires, React offre un bon équilibre entre simplicité, performance et évolutivité.

2. Back-end – Java avec Spring Boot

Rôle : Le back-end gère la logique métier, les règles de gestion, l'authentification et la communication avec la base de données.

Technologie choisie : Java + Spring Boot

Pourquoi ce choix : Java est un langage orienté objet typé et mature, dont l'architecture en classes correspond directement au diagramme UML du projet. Spring Boot structure le back-end en controllers, services et repositories, et intègre nativement Spring Security pour la gestion des rôles, et peut être configuré pour utiliser JWT via une implémentation personnalisée. La connexion à MongoDB se fait via Spring Data MongoDB.

Réponse aux besoins :

- ✓ Gestion des utilisateurs avec héritage Java : Eleve, Tuteur et Admin étendent la classe abstraite Utilisateur
- ✓ Authentification sécurisée via Spring Security et JWT
- ✓ Gestion des messages, rendez-vous et tâches via des services Spring
- ✓ Gestion de l'assignation élève → tuteur pilotée par l'Admin

Avantages :

- ✓ Typage fort et orienté objet, directement aligné avec le diagramme de classes
- ✓ Spring Security intégré pour la gestion fine des rôles et des permissions
- ✓ Large écosystème Java et documentation officielle complète et pérenne

Pourquoi pas Node.js + Express : Java Spring Boot est plus adapté aux projets qui requièrent un typage fort, une architecture orientée objet rigoureuse et une gestion de sécurité robuste, ce que reflète directement la structure du diagramme de classes UML du projet.

3. Base de données – MongoDB

Rôle : Stocker les données de l'application (utilisateurs, messages, rendez-vous, tâches).

Technologie choisie : MongoDB

Pourquoi ce choix : MongoDB est une base de données NoSQL orientée documents, adaptée aux applications web modernes.

Réponse aux besoins :

- ✓ Stockage flexible des messages et tâches
- ✓ Gestion simple des relations entre utilisateurs
- ✓ Adaptation facile aux évolutions futures

Avantages :

- ✓ Schéma flexible
- ✓ Bonne scalabilité
- ✓ Intégration native avec Spring Boot via Spring Data MongoDB

Pourquoi pas une base relationnelle : Une base SQL aurait imposé une structure plus rigide, moins adaptée à l'évolution rapide du projet.

4. Authentification – JWT (JSON Web Token)

Rôle : Sécuriser l'accès à l'application et gérer les sessions utilisateur.

Pourquoi ce choix : JWT permet une authentification stateless (sans session côté serveur), adaptée aux API REST.

Réponse aux besoins :

- ✓ Connexion sécurisée
- ✓ Protection des routes sensibles
- ✓ Gestion des rôles (élève / tuteur / admin)
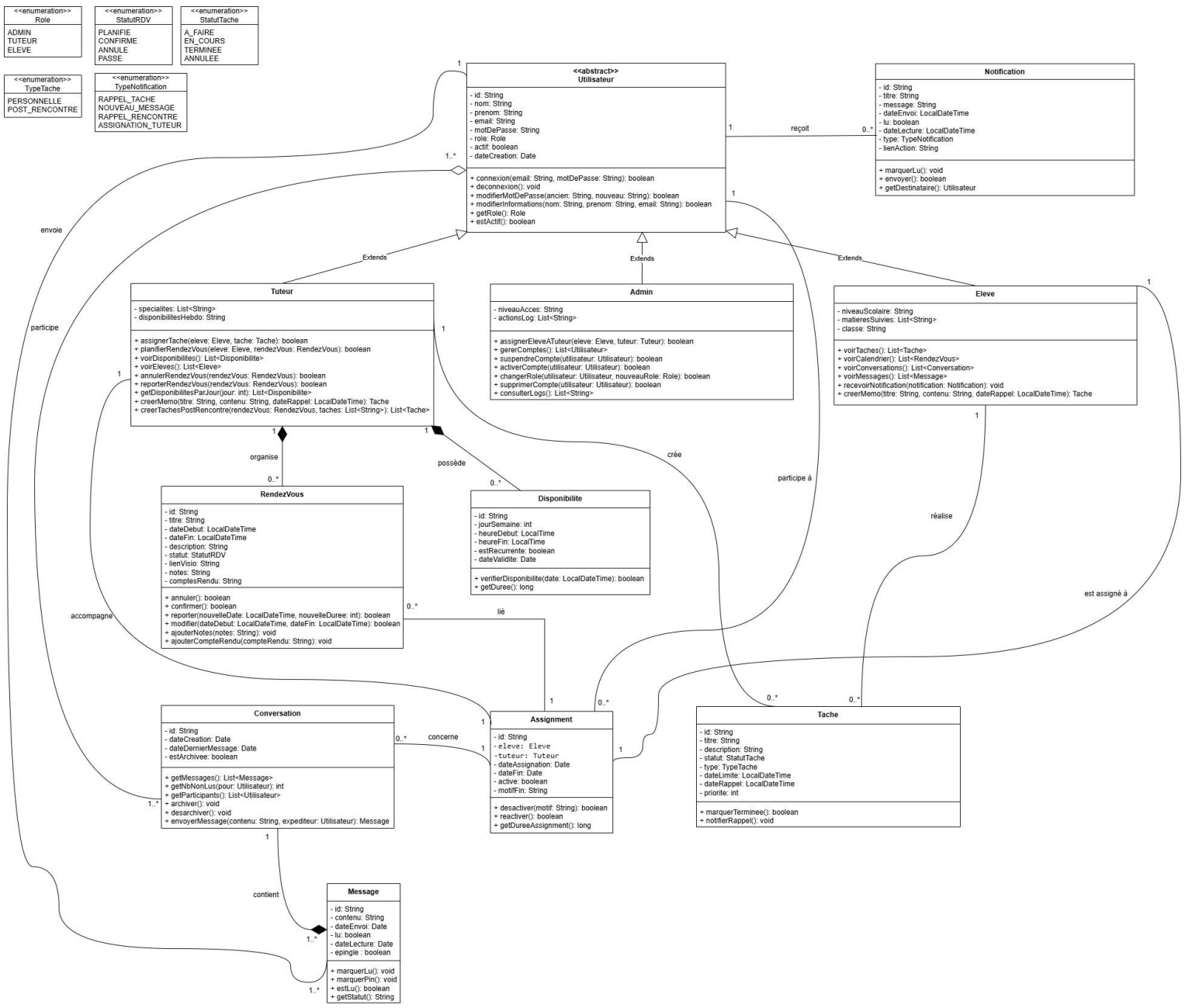
Diagramme de classes